

Informe de la Sesion 5

Modulo de Productos con Clean Architecture, MVVM, StateFlow y Koin

Rama	feature/clean-mvvm
Commit funcional	e68366b
Plataforma verificada	Android + compilacion del target iOS
Fecha	08/09/2026

Datos academicos

Curso	Desarrollo de Aplicaciones Moviles
Unidad	Fundamentos de Kotlin Multiplatform y UI con Compose Multiplatform
Estudiante	_____
Docente	_____

1. Introduccion

PharmaMobil es una aplicacion multiplataforma para la gestion farmaceutica. En las sesiones anteriores se construyeron los modelos del dominio, operaciones asincronas, formularios y navegacion. Sin embargo, el modulo Productos mantenía el estado y parte de las reglas directamente en la pantalla, utilizaba un repositorio concreto y carecia de un punto unico para ensamblar dependencias. Esta estructura dificultaba conservar el formulario ante cambios de configuracion, probar las reglas de manera aislada y sustituir el almacenamiento en memoria por un futuro backend REST.

La actividad de la Sesion 5 reorganiza este modulo mediante Clean Architecture y MVVM. El resultado separa dominio, datos y presentacion; concentra las validaciones en un caso de uso; representa toda la interfaz con un UiState inmutable; y utiliza Koin para compartir una sola instancia del repositorio en Android e iOS.

2. Objetivos

Objetivo general

Reestructurar el modulo Productos de PharmaMobil en capas de dominio, datos y presentacion, integrando ViewModel multiplataforma, StateFlow, caso de uso, repositorio en memoria e inyeccion de dependencias con Koin.

Objetivos especificos

- Definir contratos de negocio independientes de tecnologias y plataformas.
- Centralizar el registro y las validaciones de productos en RegistrarProductoUseCase.
- Conservar el estado de la pantalla en ProductoViewModel mediante un StateFlow de solo lectura.
- Modelar de forma excluyente los estados cargando, vacio, con productos y error.
- Configurar Koin en commonMain y completar su inicializacion en Android e iOS.
- Comprobar el resultado mediante compilacion, pruebas automatizadas y evidencias visuales.

3. Fundamento tecnico

Clean Architecture

La capa domain contiene modelos, contratos y casos de uso sin importar Compose, Android, iOS o mecanismos de almacenamiento. La capa data implementa dichos contratos. La capa presentation transforma eventos de usuario en cambios de estado y entrega a la vista datos listos para representar.

Modelo MVVM y StateFlow

ProductoViewModel funciona como intermediario entre la interfaz y los casos de uso. MutableStateFlow permanece privado y asStateFlow() publica una vista inmutable. viewModelScope permite ejecutar operaciones suspend sin bloquear la interfaz y cancela el trabajo cuando termina el ciclo de vida del ViewModel.

Inyeccion de dependencias con Koin

Koin ensambla ProductoRepositorioEnMemoria, RegistrarProductoUseCase y ProductoViewModel. El repositorio se declara single para que todas las pantallas trabajen con el mismo inventario. platformModule usa expect/actual para admitir configuraciones especificas de Android e iOS.

Flujo de la operacion

1	La pantalla envia nombre, precio y stock al ViewModel.
2	El ViewModel actualiza UiState y ejecuta el caso de uso.
3	El caso de uso valida los datos y solicita registrar al repositorio.
4	El repositorio asigna el ID, conserva el producto y devuelve el resultado.
5	El ViewModel recarga el inventario y publica una nueva fase de UiState.

4. Desarrollo e implementacion

4.1 Dominio

Producto incorpora la propiedad calculada `requiereReposicion` y la constante `STOCK_MINIMO = 10`. La interfaz `ProductoRepository` declara `registrar()` y `listar()` como funciones suspend en lenguaje del negocio. `DetallePedido` conserva `require(cantidad > 0)`, manteniendo su invariante en el dominio.

4.2 Datos

`ProductoRepositorioEnMemoria` implementa `ProductoRepository`, conserva una lista mutable privada, asigna identificadores correlativos y simula latencia con `delay(500)`. `listar()` devuelve una copia para impedir modificaciones externas del inventario.

4.3 Caso de uso

`RegistrarProductoUseCase` valida nombre obligatorio, precio numerico mayor que cero y stock entero no negativo. La operacion devuelve `Result`, permitiendo que presentacion trate exito y error sin depender de excepciones concretas.

4.4 Estado y ViewModel

`ProductoUiState` agrupa la fase de la lista, el formulario, los tres mensajes de error, el indicador guardando y el mensaje final. `ProductoViewModel` recibe sus dependencias por constructor, expone `StateFlow` inmutable y coordina `cargarProductos()` y `guardar()` mediante `viewModelScope`.

4.5 Interfaz e inyeccion

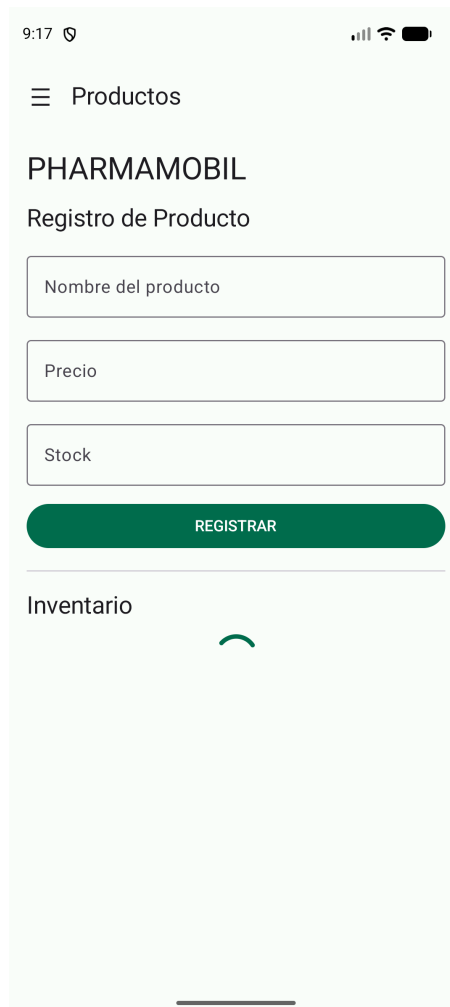
`ProductoScreen` recibe estado y callbacks, por lo que no conoce repositorios ni casos de uso. `App.kt` obtiene el `ViewModel` con `koinViewModel()`. `MainApplication` inicia Koin con `androidContext` y el manifiesto declara `android:name`. En iOS, `iOSApp.swift` invoca `initKoinlos()` antes de crear la vista.

4.6 Alcance de las evidencias

Se documentan los cuatro estados excluyentes en Android. Las variantes vacia y error se activaron temporalmente en el repositorio y se restauraron antes del commit. La ejecucion visual de iOS queda pendiente porque este entorno es Windows; no se sustituyen capturas reales por simulaciones.

5.1 Evidencia: estado cargando

Evidencia Android - Pixel 6

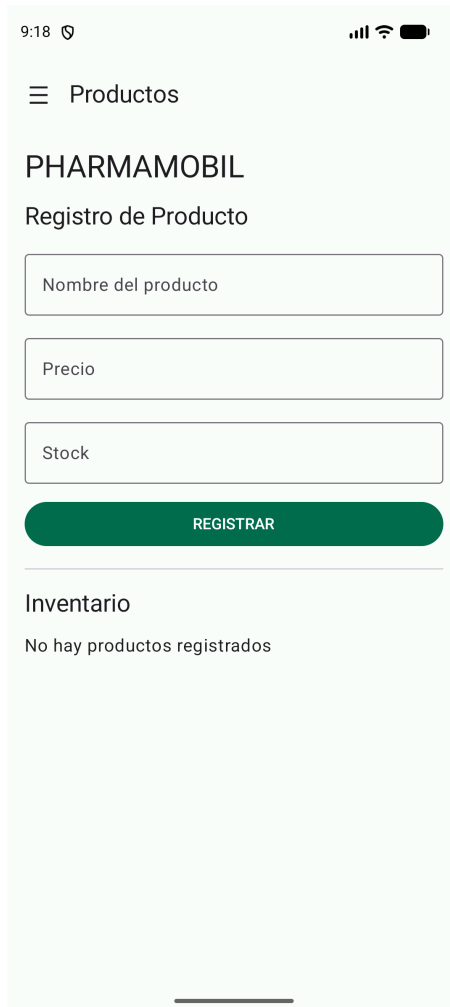


Interpretación

El indicador de progreso aparece mientras ProductoViewModel ejecuta repository.listar() dentro de viewModelScope. La fase es excluyente y no muestra simultáneamente lista, vacío o error.

5.2 Evidencia: estado sin productos

Evidencia Android - Pixel 6



9:18

Productos

PHARMAMOBIL

Registro de Producto

Nombre del producto

Precio

Stock

REGISTRAR

Inventario

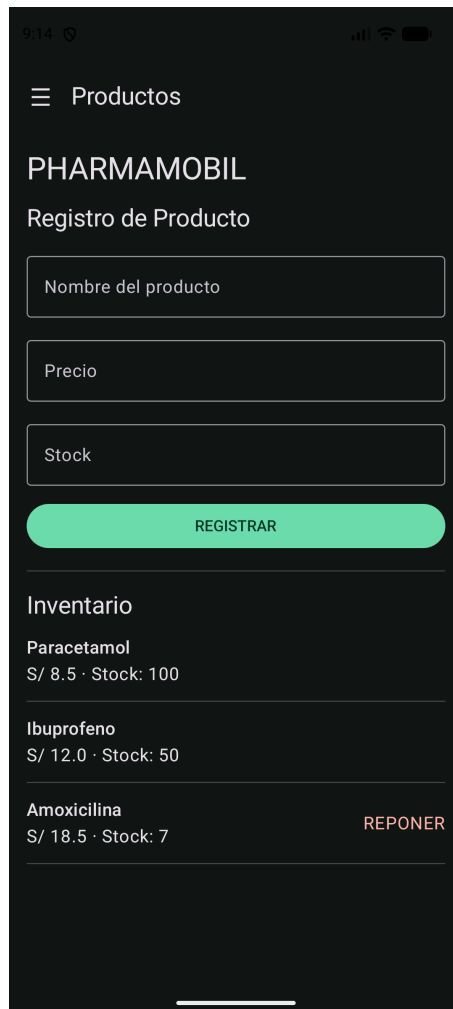
No hay productos registrados

Interpretación

Una lista vacía se transforma en `ProductoFase.SinProductos`. La pantalla resuelve la fase mediante un `when` exhaustivo y comunica el estado sin mezclar validaciones del formulario.

5.3 Evidencia: estado con productos

Evidencia Android - Pixel 6

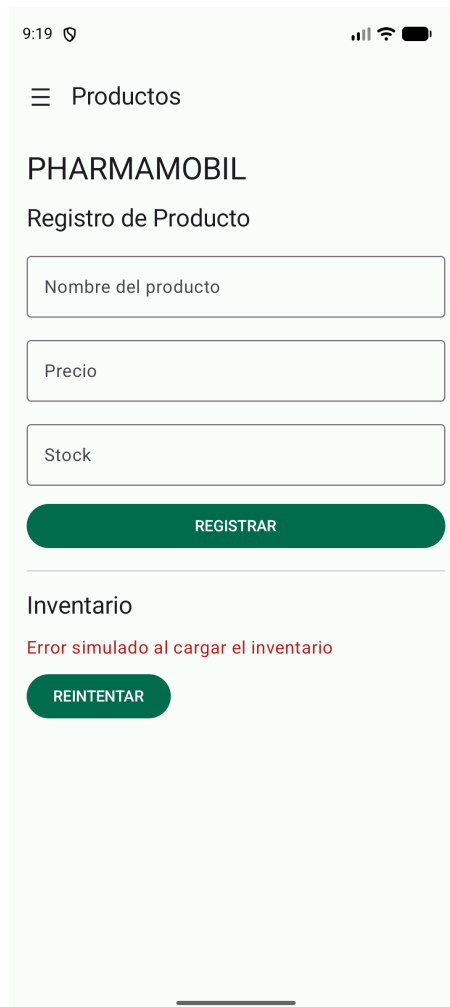


Interpretación

ProductoFase.ConProductos presenta el inventario compartido. Amoxicilina aparece con REPONER porque su stock es menor que `Producto.STOCK_MINIMO`, demostrando una regla propia del dominio.

5.4 Evidencia: estado de error

Evidencia Android - Pixel 6



Interpretación

Una excepcion del repositorio se convierte en `ProductoFase.Error`. La interfaz muestra el mensaje y ofrece `REINTENTAR`, manteniendo el fallo fuera del composible.

6. Evidencias iOS

Las cuatro capturas del simulador iOS deben obtenerse en macOS/Xcode. Este documento reserva y etiqueta cada evidencia para evitar presentar material simulado como ejecucion real.

iOS - Cargando	PENDIENTE DE CAPTURA EN macOS
iOS - Sin productos	PENDIENTE DE CAPTURA EN macOS
iOS - Con productos	PENDIENTE DE CAPTURA EN macOS
iOS - Error	PENDIENTE DE CAPTURA EN macOS

Verificacion disponible: **:shared:compileKotlinlosSimulatorArm64 - BUILD SUCCESSFUL**. La compilacion confirma que el codigo commonMain, el actual platformModule de iOS y la inicializacion initKoinlos son compatibles; no sustituye la prueba visual en Xcode.

7. Arbol de paquetes del modulo shared

La estructura separa dominio, datos, presentacion e inyeccion de dependencias.

```
shared/src
  androidMain/kotlin/pe/edu/upeu/pharmamobile/MainApplication.kt
  androidMain/kotlin/pe/edu/upeu/pharmamobile/di/PlatformModule.android.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/data/repository/ProductoRepositorioEnMemoria.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/di/AppModule.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/domain/model/Cliente.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/domain/model/DetallePedido.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/domain/model/EstadoPedido.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/domain/model/Pedido.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/domain/model/Producto.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/domain/repository/ProductoRepository.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/domain/result/ResultadoProductos.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/domain/sdsds
  commonMain/kotlin/pe/edu/upeu/pharmamobile/domain/usecase/RegistrarProductoUseCase.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/presentation/producto/ProductoScreen.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/presentation/producto/ProductoUiState.kt
  commonMain/kotlin/pe/edu/upeu/pharmamobile/presentation/producto/ProductoViewModel.kt
  commonTest/kotlin/pe/edu/upeu/pharmamobile/domain/model/ClienteTest.kt
  commonTest/kotlin/pe/edu/upeu/pharmamobile/domain/sdsd
  commonTest/kotlin/pe/edu/upeu/pharmamobile/domain/usecase/RegistrarProductoUseCaseTest.kt
  iosMain/kotlin/pe/edu/upeu/pharmamobile/di/PlatformModule.ios.kt
```

8. Ejecucion de pruebas

Comando ejecutado: `./gradlew :shared:testAndroidHostTest`

```
> Task :shared:packageAndroidMainResources UP-TO-DATE
> Task :shared:compileAndroidMainLibraryResources UP-TO-DATE
> Task :shared:parseAndroidMainLocalResources UP-TO-DATE
> Task :shared:generateAndroidMainRFile UP-TO-DATE
> Task :shared:checkAndroidHostTestAarMetadata UP-TO-DATE
> Task :shared:mergeAndroidHostTestAssets UP-TO-DATE
> Task :shared:mergeAndroidHostTestResources UP-TO-DATE
> Task :shared:mergeAndroidHostTestManifest UP-TO-DATE
> Task :shared:processAndroidHostTestManifest UP-TO-DATE
> Task :shared:processAndroidHostTestResources UP-TO-DATE
> Task :shared:packageAndroidHostTestForUnitTest UP-TO-DATE
> Task :shared:generateAndroidHostTestConfig UP-TO-DATE
> Task :shared:compileAndroidMain UP-TO-DATE
> Task :shared:bundleAndroidMainClassesToRuntimeJar UP-TO-DATE
> Task :shared:bundleAndroidMainClassesToCompileJar UP-TO-DATE
> Task :shared:processAndroidMainJavaRes UP-TO-DATE
> Task :shared:compileAndroidHostTest UP-TO-DATE
> Task :shared:testAndroidHostTest UP-TO-DATE

BUILD SUCCESSFUL in 1s
34 actionable tasks: 1 executed, 33 up-to-date
Configuration cache entry reused.
```

Resultado: APROBADO (codigo de salida 0).

9. Lista de cotejo

Criterio	Resultado
Paquetes Clean	Cumple
Dominio independiente de plataforma	Cumple
Regla requiereReposicion	Cumple
Repositorio interfaz + memoria suspend	Cumple
ID asignado en repositorio	Cumple
Caso de uso devuelve Result	Cumple
UiState con cuatro fases	Cumple
StateFlow de solo lectura	Cumple
ProductoScreen sin estado de negocio	Cumple
Koin con repositorio single	Cumple
Inicializacion Android/iOS	Cumple por compilacion
Capturas Android/iOS	Android cumple; iOS pendiente
Pruebas y rama versionada	Cumple

10. Conclusiones

El modulo Productos opera de extremo a extremo con responsabilidades separadas: la vista representa ProductoUiState, el ViewModel coordina operaciones asincronas, el caso de uso concentra reglas y el repositorio conserva el inventario y asigna identificadores. Koin mantiene una unica instancia del repositorio. La entrega tecnica queda completa; para cerrar al 100% la evidencia multiplataforma solo falta ejecutar Xcode en macOS y reemplazar los cuatro recuadros pendientes por capturas reales.